

Data Structure Week 5

Advance Programming

Steven Wijaya
Informatika (Kampus Kota Makassar)
Universitas Ciputra Makassar
swijaya08@student.ciputra.ac.id

I. QUESTION 1 – BRACKET BALANCER

A. Ringkasan Masalah

Program ini bertujuan untuk memeriksa apakah sebuah urutan tanda kurung (brackets) seimbang atau tidak. Syarat keseimbangan dalam program ini dikhususkan untuk string tanda kurung bersarang yang bersifat simetris atau seperti cermin (misalnya: $\{([])\}$).

B. Desain Stack

Apa yang di-push/popped dan mengapa: Karakter tanda kurung dibelah menjadi dua bagian. Setengah bagian pertama di-push ke dalam stack1, dan setengah bagian sisanya di-push ke dalam stack2. Keduanya kemudian di-pop secara bersamaan untuk dicocokkan.

Representasi state: Stack merepresentasikan dua sisi dari string yang dibelah. stack1 menyimpan sisi kiri, dan stack2 menyimpan sisi kanan yang masuk secara LIFO, sehingga saat di-pop, urutan dari tengah (pusat cermin) akan keluar lebih dulu menuju ke luar.

C. Penjelasan Algoritma

Langkah-demi-langkah:

1. Periksa panjang string. Jika ganjil, otomatis tidak seimbang (return false).
2. Cari titik tengah string.
3. Looping pertama memasukkan karakter dari indeks 0 hingga tengah ke stack1.
4. Looping kedua memasukkan karakter dari titik tengah hingga akhir ke stack2.
5. Selama stack belum kosong, pop 1 karakter dari stack1 dan 1 dari stack2.
6. Cek menggunakan fungsi bantuan isMatchingPair. Jika ada satu saja yang tidak cocok, kembalikan false. Jika semua lolos, kembalikan true.

Edge Cases: String dengan panjang ganjil akan langsung ditolak untuk menghindari iterasi yang sia-sia atau error out-of-bounds. String kosong akan dianggap balanced.

D. Argumen Kebenaran & Twist Requirement

Algoritma ini menghasilkan output yang benar khusus untuk twist requirement (persyaratan unik) di mana urutan bracket wajib berupa cermin bersarang penuh (contoh: $\{([])\}$). Pendekatan konvensional biasanya hanya menggunakan 1 stack untuk mencocokkan buka-tutup di posisi mana pun. Namun, dengan membelah string ke dalam 2 stack, kode ini memaksa struktur harus benar-benar simetris dari tengah ke luar, memenuhi twist dari soal ini.

E. Kompleksitas Waktu & Ruang Waktu

$O(N)$ di mana N adalah panjang string. Kita melakukan iterasi string secara linier untuk melakukan proses push dan mem-pop semuanya dalam satu iterasi while.

F. Bukti Pengujian

```
=== Program Cek Bracket Balanced ===
Masukkan urutan bracket (contoh: {[()]}) : ([{}])
Hasil: NOT BALANCED (Tidak Seimbang)
PS C:\Users\steve\Documents\ProjectJava\DataStructure5>
```

G. Referensi Kode

[ds-stack-StevenWijaya-2086022510021/src/Question1.java](https://github.com/StevenWijaya-2086022510021/src/Question1.java) at main · StevenWijaya-2008/ds-stack-StevenWijaya-2086022510021

II. QUESTION 2 – RPN

A. Ringkasan Masalah

Program ini bertugas untuk mengevaluasi ekspresi matematika yang ditulis dalam format *Reverse Polish Notation* (Postfix). Dalam RPN, operator selalu diletakkan setelah kedua operan (contoh: $2\ 1\ +$ berarti $2 + 1$).

B. Desain Stack

Apa yang di-push/popped dan mengapa: Angka (operan) di-push ke dalam Stack bertipe Integer. Ketika program menemukan token berupa operator (+, -, *, /), program akan mem-pop dua angka teratas dari stack, melakukan perhitungan matematis, dan langsung mem-push kembali hasilnya ke dalam stack.

Representasi state: Stack selalu menyimpan angka-angka tunggal atau hasil kalkulasi sementara yang sedang menunggu operator berikutnya.

C. Penjelasan Algoritma

Langkah-demi-langkah:

1. Iterasi setiap token dalam array of string.
2. Jika token adalah angka, ubah ke integer dan masukkan (push) ke stack.
3. Jika token adalah operator matematis, pop angka pertama ke variabel b dan pop angka kedua ke variabel a .
4. Hitung a [operator] b , lalu push hasilnya.
5. Setelah semua token habis, hasil akhir adalah satu-satunya elemen yang tersisa di dalam stack (di-pop lalu di-return).

Edge Cases: Pembagian oleh nol (walaupun diasumsikan input valid). Pengurangan dan pembagian harus berhati-hati dalam urutan pop, elemen yang pertama di-pop adalah bilangan pembagi/pengurang.

D. Argumen Kebenaran & Twist Requirement

Algoritma ini menghasilkan output yang akurat karena sifat LIFO dari stack secara alami sangat cocok untuk struktur Postfix tree. Dua angka yang masuk terakhir adalah yang paling berhak untuk dioperasikan oleh operator yang baru saja muncul. Perlakuan pemisahan b (pop pertama) dan a (pop kedua) menjamin bahwa ekspresi tak-komutatif seperti $a - b$ dan a / b dieksekusi dengan urutan kiri-kanan yang benar, tidak terbalik.

E. Kompleksitas Waktu & Ruang Waktu

$O(N)$ di mana N adalah jumlah token dalam array input. Setiap token diproses tepat satu kali.

F. Bukti Pengujian

```
Input 1: 2 1 + 3 *
Output 1: 9
Input 2: 4 13 5 / +
Output 2: 6
PS C:\Users\steve\Documents\ProjectJava\DataStructure5>
```

G. Referensi Kode

[ds-stack-StevenWijaya-2086022510021/src/Question1.java](https://github.com/StevenWijaya-2086022510021/src/Question1.java) at main · StevenWijaya-2008/ds-stack-StevenWijaya-2086022510021

III. QUESTION 3 – CHRONO STACK ENGINE

A. Ringkasan Masalah

Dalam simulasi mesin fiksi Chrono Stack Engine, kita diharuskan mencari kombinasi instruksi minimum (1, d, +) untuk membentuk timeline tumpukan angka yang diinginkan dari bawah ke atas. Efek samping mematikan: setiap pemanggilan operasi penggabungan (+) akan mereduksi/mendistorsi seluruh angka di bawahnya sebanyak 1..

B. Desain Stack

Apa yang di-push/popped dan mengapa:

1. targetStack (tipe Long): Menyimpan input target awal. Di-pop dari atas ke bawah untuk menghitung akumulasi "pajak" distorsi.
2. codeStack (tipe String): Menyimpan instruksi kode seperti $1d+d+$. Karena instruksi harus dicetak dari dasar stack ke atas, sifat LIFO stack ini akan membalik kembali kodenya menjadi urutan cetak yang benar.

Representasi state: currentTax menyimpan akumulasi jumlah instruksi + masa depan yang mengancam timeline di bawahnya. Stack bertindak sebagai mesin penjelajah waktu yang menghitung dari hasil akhir kembali ke awal.

C. Penjelasan Algoritma

Langkah-demi-langkah:

1. Masukkan semua target angka ke targetStack.
2. Lakukan iterasi while dengan mem-pop elemen teratas targetStack.
3. Tambahkan target tersebut dengan akumulasi pajak (currentTax) dari iterasi sebelumnya. Inilah nilai asli yang harus dibangun.
4. Ubah angka menjadi bentuk binary string (basis 2).
5. Parse binary tersebut: angka selalu diawali 1. Setiap geser digit tambah instruksi duplicate lalu merge

(d+). Jika digit biner saat itu adalah '1', beri ekstra add 1 (1+).

6. Simpan string kode ke codeStack dan tambahkan total tombol + yang barusan tercipta ke variabel currentTax.
7. Pop codeStack sampai habis untuk membalik urutan teks kembali dari bawah ke atas dan cetak.

Edge Cases: Output kompensasi dapat membengkak menjadi miliaran akibat efek akumulatif operasi +. Oleh karena itu tipe data long digunakan untuk mencegah Integer Overflow (angka menjadi negatif).

D. Argumen Kebenaran & Twist Requirement

Twist utamanya adalah instruksi + mengganggu angka yang telah dicetak sebelumnya ("mengganggu masa lalu"). Kode ini memecahkan masalah tersebut dengan melakukan algoritma yang berjalan mundur (dari top stack ke bottom stack). Sifat LIFO dari targetStack memungkinkan kita menghitung akumulasi instruksi + yang akan terjadi di masa depan, dan menggunakannya sebagai subsidi kompensasi untuk elemen-elemen di masa lalu, sehingga pada saat eksekusi sesungguhnya dari bawah ke atas, angka yang besar tersebut akan secara presisi terdistorsi tepat menjadi nilai target akhirnya.

E. Kompleksitas Waktu & Ruang Waktu

$O(N \cdot B)$ di mana N adalah jumlah elemen dan B adalah jumlah bit biner maksimum (maksimal 64 untuk tipe Long). Secara teknis dapat dianggap $O(N)$..

F. Bukti Pengujian

```
5
1 5 6 3 2
1d+1d+d+1d+1d+d+1d+d+1d+d+1d+d+1d+d+1d+d+
PS C:\Users\steve\Documents\ProjectJava\DataStructure5>
```

G. Referensi Kode

[ds-stack-StevenWijaya-2086022510021/src/Question3.java](https://github.com/StevenWijaya-2086022510021/src/Question3.java) at main · StevenWijaya-2008/ds-stack-StevenWijaya-2086022510021

IV. QUESTION 4 – STACK SORTING

A. Ringkasan Masalah

Tujuannya adalah mengurutkan elemen integer yang diinput dalam satu baris (dipisah dengan spasi) menjadi urutan ascending (terkecil ke terbesar) menggunakan mekanisme murni Stack, tanpa memanfaatkan struktur array atau metode sorting bawaan seperti Collections.sort().

B. Desain Stack

Apa yang di-push/popped dan mengapa:

1. mainStack: Bertindak sebagai tempat penyimpanan elemen-elemen yang kondisinya selalu dijaga tetap urut.
2. tempStack: Bertindak sebagai penyangga sementara (bantuan). Jika ada elemen baru dari input yang lebih kecil dari elemen pucuk mainStack, elemen-elemen pucuk tersebut di-pop dan diamankan ke tempStack terlebih dulu.

Representasi state: Kombinasi kedua stack ini mensimulasikan mekanisme Insertion Sort di mana data ditarik-ulur untuk mencari posisi sisip yang pas bagi elemen baru.

C. Penjelasan Algoritma

Langkah-demi-langkah:

1. Baca satu baris string dan pecah berdasarkan spasi menjadi sebuah array of token string.
2. Iterasi setiap token dan ubah menjadi integer.
3. Saat akan memasukkan angka baru ke mainStack: Selama stack tidak kosong dan angka teratas mainStack lebih besar dari angka baru, pindahkan angka tersebut ke tempStack.
4. Masukkan angka baru ke mainStack yang posisinya sudah disiapkan.
5. Kembalikan semua elemen di tempStack ke mainStack.
6. Di akhir proses, mainStack memiliki elemen urut terbalik (descending). Untuk mencetak secara ascending, pop seluruh mainStack kembali ke tempStack, lalu pop dan cetak tempStack.

D. Argumen Kebenaran & Twist Requirement

Twist di soal ini adalah pelarangan penggunaan algoritma sorting standar dan batasan hanya menggunakan struktur Stack. Algoritma menyelesaikannya dengan memastikan invariant dari mainStack: isinya harus selalu dalam keadaan terurut (descending) setiap kali proses perpindahan dari/ke tempStack selesai dilakukan. Sehingga di tahap finalisasi pengeluaran, cetakan otomatis terurut kecil ke besar.

E. Kompleksitas Waktu & Ruang Waktu

$O(N^2)$ pada skenario terburuk, karena setiap angka baru mungkin mengharuskan kita memindahkan seluruh isi mainStack ke tempStack dan mengembalikannya lagi.

F. Bukti Pengujian

```
1 5 3 19 31 9
1 3 5 9 19 31
PS C:\Users\steve\Documents\ProjectJava\DataStructure5>
```

G. Referensi Kode

[ds-stack-StevenWijaya-2086022510021/src/Question4.java](https://github.com/StevenWijaya-2086022510021/src/Question4.java) at main · StevenWijaya-2008/ds-stack-StevenWijaya-2086022510021

C. Penjelasan Algoritma

Langkah-demi-langkah:

1. Minta input nilai awal (momentum) N.
2. Masukkan inisiasi nilai current dengan N.
3. Selama nilai current ≥ 1 : push nilai tersebut ke Stack, lalu mutasikan nilai current menjadi current / 2 (pembagian integer, koma diabaikan).
4. Setelah perulangan selesai, pucuk (top) Stack pasti bernilai 1.
5. Lakukan iterasi mem-pop Stack sampai habis untuk dicetak, yang akan menghasilkan pola dari 1 ke N.

Edge Cases: Apabila input awal adalah angka negatif atau 0, kondisi while tidak akan tereksekusi sehingga Stack tetap kosong dan program berakhir tanpa melempar pengecualian (error).

D. Argumen Kebenaran & Twist Requirement

Mekanisme yang paling mudah untuk mencetak mundur sesuatu yang dihasilkan secara sekuensial adalah dengan fungsi rekursi atau Array Reverse. Namun dengan mematuhi syarat penggunaan struktur data Stack (LIFO), twist ini ditangani secara elegan dan seketika. Elemen terakhir yang dihasilkan dari pembagian (angka 1) adalah elemen yang paling akhir dimasukkan ke stack, sehingga menjamin saat proses pop berlangsung, angka 1 keluar lebih awal dan naik berlipat ganda sesuai hierarki output yang diinginkan.

E. Kompleksitas Waktu & Ruang Waktu

$O(\log N)$ karena perulangan while membagi nilai N dengan 2 pada tiap langkahnya secara konstan..

F. Bukti Pengujian

```
50
1 3 6 12 25 50
PS C:\Users\steve\Documents\ProjectJava\DataStructure5>
```

G. Referensi Kode

[ds-stack-StevenWijaya-2086022510021/src/Question5.java](https://github.com/StevenWijaya-2086022510021/src/Question5.java) at main · StevenWijaya-2008/ds-stack-StevenWijaya-2086022510021

V. QUESTION 5 – MOMENTUM SEQUENCE

A. Ringkasan Masalah

Diberikan sebuah nilai momentum awal N. Program harus mencetak sequence pergerakan angka mulai dari N yang terus dibagi 2 hingga mencapai angka 1. Uniknya, rentetan angka ini harus dicetak dari yang terkecil (1) ke yang terbesar (momentum awal N).

B. Desain Stack

Apa yang di-push/popped dan mengapa: Angka integer N, diikuti oleh hasil pembagian dua dari integer division berturut-turut ($N/2$, $N/4$, ..., 1) akan di-push ke dalam satu Stack utama secara berurutan. Saat di-pop, mereka akan langsung dicetak.

Representasi state: Stack menyimpan rekam jejak historis dari hasil pembagian angka tersebut secara berurutan dari angka raksasa menuju 1.

